

Phase 7: Project Documentation Phase

Project Name: **FitSense (FitTrack AI)** — System Specifications, API Reference & Operational Manual

7.1 Documentation Overview & Project Deliverables Prepared

The project documentation phase synthesizes all engineering artifacts, deployment protocols, API interface definitions, and developer instructions for the FitSense (AI FitTrack) backend system. Delivered materials include full technical documentation, user guidance via REST client workflows, reproducible installation steps, environment configurations, and an exhaustive REST endpoint reference manual.

7.2 System Prerequisites & Technical Baseline

System Dependencies

- **Operating System:** Windows 10/11, macOS, Linux (cross-platform compatible).
- **Node.js:** Version 16 or above (asynchronous V8 engine).
- **npm:** Package Manager Version 8 or above.
- **Database Engine:** MongoDB (community or enterprise server instance).
- **Testing Client:** Postman or VS Code Thunder Client extension.

Hardware Baseline

- **CPU:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- **System RAM:** 8 GB minimum (16 GB recommended for concurrent instances of Node, processing layers, and MongoDB).
- **Disk Space:** 1 GB available disk workspace.
- **Network:** Active internet connection for Google Gemini AI endpoints.

7.3 Step-by-Step Installation & Setup Instructions

Follow the sequence documented in Epic 2 and Epic 5 to install and run the application:

1. **Open Workspace:** Launch Visual Studio Code and enter the project repository: `cd AI-FitTrack-API/server`.
2. **Install Dependencies:** Run `npm install express mongoose dotenv bcryptjs jsonwebtoken cors`.
3. **Install Development Daemon:** Run `npm install --save-dev nodemon`.
4. **Configure Environment Secrets:** Create a `.env` file in the server root folder containing:

```
PORT=5000 MONGO_URI=mongodb://localhost:27017/aifittrack JWT_SECRET=your_secret_key
GEMINI_API_KEY=your_google_gemini_api_key
```

5. **Start Application Server:** Execute `npm start` in terminal. Express binds to port 5000 and connects to MongoDB.

7.4 Architecture & Module Documentation Summary

Module Tier	Directory / File	Documented Operational Scope
Server Entry	server.js / app.js	Initializes Express, enables CORS, parses incoming JSON payloads, registers API route trees, and captures centralized errors.
Database Config	config/db.js	Configures Mongoose ODM connection to MongoDB URI, logs host connection details, and exits process on critical failure.
Auth Middleware	middleware/auth.js	Inspects Authorization: Bearer <token>, checks signature against JWT_SECRET, and injects decoded user to req.user.
Controllers	controllers/*.js	Executes domain logic for authController.js, workoutController.js, and aiController.js.

7.5 API Interface Specification Manual

All API endpoints return standardized JSON payloads. Protected endpoints mandate a valid JWT Bearer token.

Method	Endpoint Path	Access Level	Request Schema / Parameters	Response Summary
POST	/api/auth/register	Public	JSON: { name, email, password }	Status 201; JWT token & user object.
POST	/api/auth/login	Public	JSON: { email, password }	Status 200; JWT token & user object.
GET	/api/auth/profile	Bearer JWT	Headers: Authorization: Bearer <token>	Status 200; User identity record.
POST	/api/workouts/	Bearer JWT	JSON: { workoutName, category, duration, caloriesBurned, workoutDate }	Status 201; Created workout document.
GET	/api/workouts/	Bearer JWT	Headers: Authorization: Bearer <token>	Status 200; Workout array & total count.
GET	/api/workouts/:id	Bearer JWT	URL Param: :id (Workout ObjectId)	Status 200; Targeted workout document.
PUT	/api/workouts/:id	Bearer JWT	JSON: Fields to update (e.g., duration, calories)	Status 200; Updated workout object.
DELETE	/api/workouts/:id	Bearer JWT	URL Param: :id (Workout ObjectId)	Status 200; "Workout removed successfully".
GET	/api/workouts/search	Bearer JWT	Query params: workoutName, category, workoutDate	Status 200; Filtered workout array.
POST	/api/ai/workout-recommendation	Bearer JWT	JSON: { age, fitnessGoal, experience }	Status 200; Generated workout plan & safety tips.
POST	/api/ai/fitness-insights	Bearer JWT	JSON: { totalWorkouts, averageDuration, totalCaloriesBurned }	Status 200; AI analysis & improvement suggestions.

7.6 User Documentation & Workflow

- Create user account via /api/auth/register.
- Log in via /api/auth/login and capture token string.
- Attach token in client headers: Authorization: Bearer <token>.
- Submit workout logs, execute updates, and run queries.
- Submit user goals to AI endpoints to receive custom coaching routines.

7.7 System Maintenance Guidelines

- Credential Management:** Periodically rotate JWT_SECRET and GEMINI_API_KEY within the .env file.
- Database Health:** Maintain MongoDB indexes on user, category, and workoutDate fields for high search performance.
- Error Monitoring:** Centralized error middleware captures validation and unhandled exceptions to prevent thread crashes.

7.8 Documented Limitations

- Headless REST backend without bundled graphical front-end templates.
- Single user role model without specialized administrative panels.
- AI execution depends on external Google Gemini endpoint availability.
- Nutrition and wearable hardware ingestion are outside the present scope.

7.9 Future Enhancements

- Extend Mongoose schemas to incorporate nutrition tracking and diet logging.
- Implement automated Body Mass Index (BMI) calculating utilities.
- Integrate smart wearable devices (smartwatches) for automated heart-rate capture.
- Develop automated training reminders and graphical web/mobile dashboards.